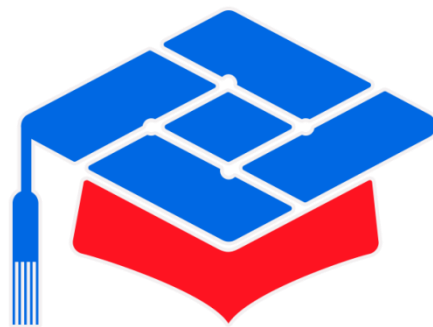




第九届

全国大学生集成电路创新创业大赛

报告类型： 技术文档

参赛杯赛： 海云捷讯杯

作品名称： 基于 FPGA 的机械臂精确控制系统设计

队伍编号： CICC0906044

团队名称： 泓杰大队

目录

1 系统框架介绍	3
1.1 系统框图.....	3
1.2 图像采集模块.....	3
1.3 图像处理模块.....	4
1.4 机械臂控制模块.....	5
1.5 任务控制模块.....	5
2 关键技术分析	6
2.1 图像显示.....	6
2.2 图像处理.....	8
2.2.1 RGB 色彩模型.....	8
2.2.2 HSV 色彩模型	8
2.2.3 二值化.....	10
2.2.4 形态学滤波-开操作	11
2.2.5 目标识别.....	12
2.2.6 角度信息获取与坐标转换.....	13
2.3 机械臂控制.....	15
2.3.1 逆运动学解算.....	15
2.3.2 仓库位置机械臂关节对应角度的存储.....	16
2.3.3 减少误差设计.....	17
2.3.4 角度偏移处理.....	19
2.3.5 单个吸取状态机设计.....	20
2.4 任务控制.....	21
2.4.1 目标位置信息传递.....	21
2.4.2 跨时钟域处理.....	22
2.5 上位机设计与控制.....	24
2.5.1 软件架构设计.....	24
2.5.2 核心功能实现.....	24
2.5.3 关键技术实现.....	25
2.5.4 错误处理与异常机制.....	26
2.5.5 使用指南.....	26
3 性能分析	27
3.1 目标锁定时间.....	27
3.1.1. 图像采集与预处理耗时分析.....	27
3.1.2. 形态学处理耗时分析.....	27
3.1.3. 目标检测与形状识别耗时分析.....	27
3.1.4. 坐标转换与输出耗时分析.....	27
3.1.5. 整体分析耗时分析.....	27
3.2 吸取准确度与仓库放置准确度.....	28
3.3 移动速度.....	28

1 系统框架介绍

1.1 系统框图

整个工作系统可以简略分为四个模块，分别是 ov5640 图像采集，图像处理模块，机械臂控制模块，任务控制模块（用来联系图像处理模块和机械臂处理模块），下面分别对这四个部分进行展开说明

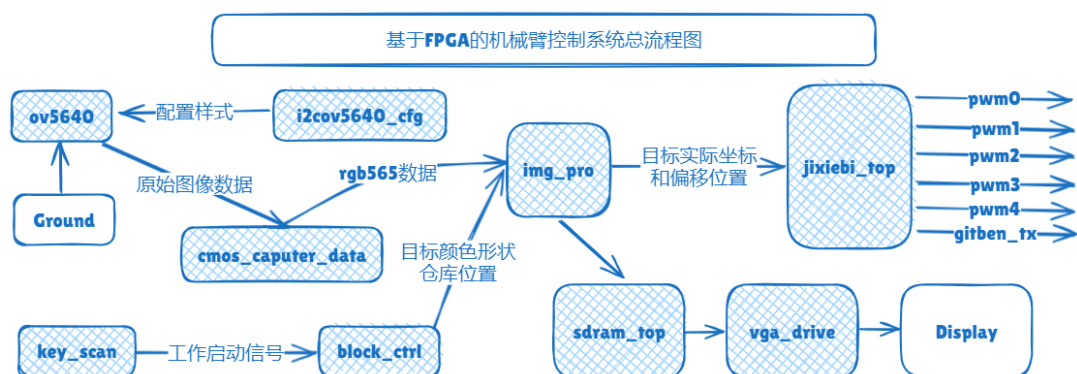


图 1.1-1 基于 FPGA 的机械臂控制系统总流程图

1.2 图像采集模块

OV5640 是一款高性能 500 万像素 CMOS 图像传感器，采用 $1.4\mu\text{m}$ 像素尺寸和 OmniBSI 技术，支持自动曝光、白平衡等图像控制功能，可输出 RGB565 或 JPEG 格式图像。

本设计通过修改正点原子例程，配置其输出 $640\times 480@60\text{Hz}$ RGB565 格式。系统采用 SDRAM 乒乓缓冲机制，将 16 位 256Mb SDRAM 划分为双 Bank，交替存储和读取图像数据，有效解决高速采集与显示间的速率匹配问题。

VGA 显示部分基于标准 25.175MHz 时序，将 RGB565 数据通过数模转换生成模拟信号，配合行场同步信号实现逐行扫描显示。该方案完整实现了从图像采集、缓冲到显示的实时处理流程。



图 1.2-1 OV5640 图像传感器



图 1.2-2 图像采集工作流程图

实际采集图像如下



图 1.2-3 VGA 采集图像

1.3 图像处理模块

在颜色识别任务中，RGB 模型的表现往往不如其它颜色模型，例如 HSV 或者 Lab。例如，当光照条件变化时，一个物体的颜色可能会在 RGB 空间中出现显著的偏移，导致识别困难。这就需要采用更为鲁棒的颜色空间，或者配合其它颜色处理技术来提高颜色识别的准确度和稳定性。

因此，本项目采用先将 rgb 转换成 hsv 色彩后进行颜色识别。得到 hsv 色彩后进行根据目标颜色进行二值化处理得到二值化数据，然后进行形态学滤波，先进行腐蚀操作，再进行膨胀操作，得到更加平滑噪声更少的图像后进行目标识别获得框住目标的四个顶点坐标。

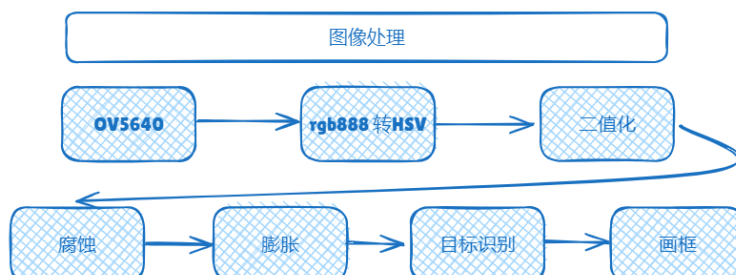


图 1.3-1 图像处理流程图

1.4 机械臂控制模块

机械臂控制模块在接受到图像里模块传来的实际坐标后和目标仓库位置后执行动作，一套动作包括逆运动学解算，气泵控制模块，获取目标仓库位置所对应各个关节所运动的关节，这里由于目标仓库的位置是定死的，为了节省资源我将目标仓库的 24 个槽位设计成查找表的形式，输入为 0~23 索引地址，输出是对应关节所转动的角度。

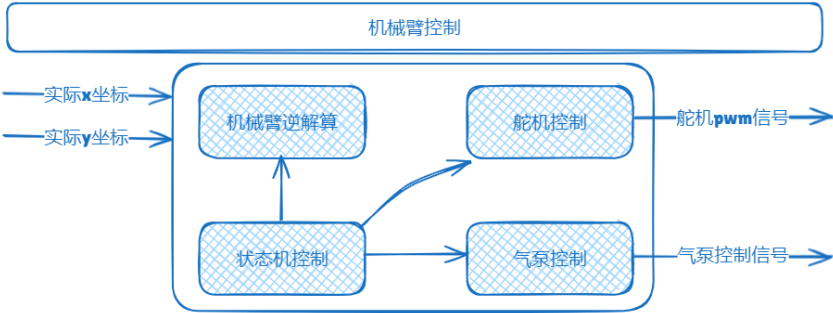


图 1.4-1 机械臂控制流程图

1.5 任务控制模块

该模块用来联系图像信息采集和机械臂运动，使系统正确运作，题目要求是将目标场地的物块使用机械臂吸取将物块移动到仓库的相应位置，我的做法是让仓库来发出请求工作，使用二维数组记录每个槽位的信息（有无物块防止，物块颜色，物块角度），在任务控制模块里查询仓库信息发出视觉请求给图像处理模块，发出目标仓库位置索引给机械臂控制模块，再接受到机械臂控制模块单个物块移动完成的信号后，接着查询下一个需要放置物块的槽位，如此反复，直至到最后一个槽位后，一次工作完成。

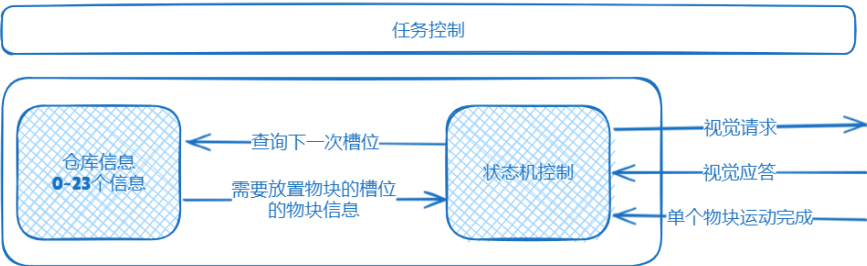


图 1.5-1 任务控制流程图

2 关键技术分析

2.1 图像显示

1. 通过 SCCB 协议配置 ov5640 摄像头后，OV5640 摄像头在 RGB565 模式下输出 16 位数据（R[4:0]+G[5:0]+B[4:0]）。为了节省引脚，只提供 8 位数据总线，因此需要分两次传输（先高 8 位，后低 8 位）。摄像头输出八位 rgb 数据，我们需要接受俩次数据进行拼接得到一个完整的 rgb565 数据。代码实现如下：

```
//8位数据转16位RGB565数据
always @(posedge cam_pclk or negedge rst_n) begin
    if(!rst_n) begin
        cmos_data_t <= 16'd0;
        cam_data_d0 <= 8'd0;
        byte_flag <= 1'b0;
    end
    else if(cam_href) begin
        byte_flag <= ~byte_flag;
        cam_data_d0 <= cam_data;
        if(byte_flag)
            cmos_data_t <= {cam_data_d0,cam_data};
        else;
    end
    else begin
        byte_flag <= 1'b0;
        cam_data_d0 <= 8'b0;
    end
end
end
```

图 2.1-1 图像数据拼接代码

2. 由于摄像头持续采集的图像数据速率（约 14.7MB/s）与 VGA 显示读取的速率需要精确匹配，系统采用了先进的乒乓缓存机制。该机制将 SDRAM 物理存储空间划分为两个逻辑存储区（Bank A 和 Bank B），通过巧妙的状态机控制实现读写操作的交替进行。

具体工作流程是：当摄像头数据正在写入 Bank A 时，VGA 控制器从 Bank B 读取数据；当一帧图像采集完成后，立即切换为摄像头数据写入 Bank B，同时 VGA 从 Bank A 读取。这种双缓冲结构完全避免了读写冲突，确保了数据的完整性和显示的流畅性。采用的正点原子历程实现乒乓操作的关键代码如下

```
if(sdrdram_pingpang_en) begin //若未到达写SDRAM的结束地址，则写BankA
    if(sdrdram_wr_addr[22:0] < wr_max_addr - wr_length) //SDRAM 读写乒乓使能
        sdrdram_wr_addr <= sdrdram_wr_addr + wr_length;
    else begin //切换BANK
        rw_bank_flag <= ~rw_bank_flag;
        sw_bank_en <= 1'b1; //拉高切换BANK使能信号
    end
end
end //若突发写SDRAM结束，更改写地址
```

图 2.1-2 乒乓操作代码

3. 在显示输出端，VGA 控制器需要精确生成符合 VGA 标准的时序信号。对于 $640 \times 480 @ 60\text{Hz}$ 的分辨率，系统使用 25.175MHz 的像素时钟（由锁相环 ip 核生成），每个完整的行周期包含 800 个时钟周期，其中 640 个用于有效像素显示，160 个为行消隐期；每个完整的场周期包含 525 行，其中 480 行为有效显示行，45 行为场消隐期。

控制器从 SDRAM 中读取的 RGB565 数据需要通过简单的电阻网络实现数模转换，生成模拟的 VGA 信号。同时，严格按照标准生成的 HSYNC 和 VSYNC 同步信号确保了显示设备的正确扫描。

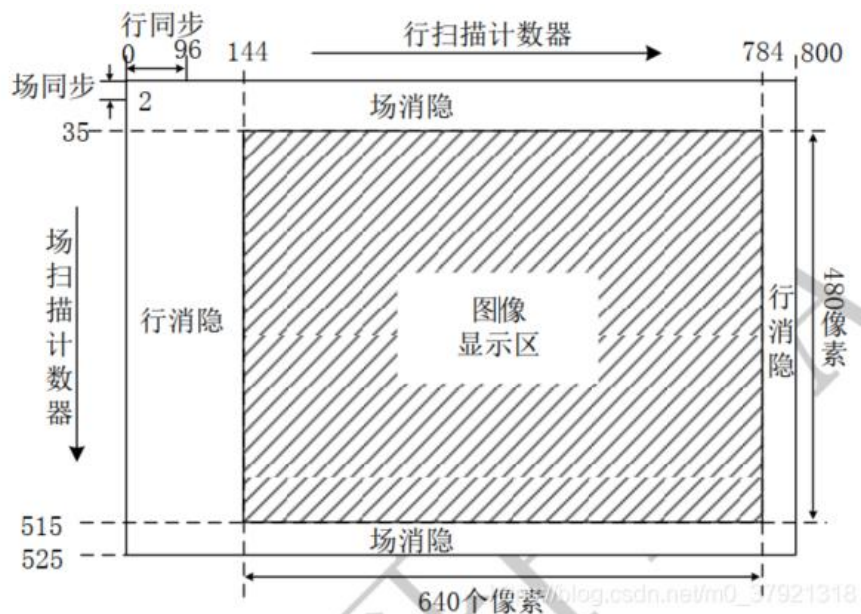


图 2.1-3 VGA 时序图解

另外为方便图像坐标系和机械臂坐标系的转换，在显示器的中心位置我们增加了一个十字，与场地中心进行匹配。



图 2.1-4 十字匹配标识

2.2 图像处理

2.2.1 RGB 色彩模型

RGB 色彩空间使用 Red 红色、Green 绿色、Blue 蓝色三个分量来表示颜色，每个分量的范围是[0%, 100%]。在计算机中，大部分使用 RGB 色彩空间的程序都使用 1 字节来存储其颜色分量，这时每个分量的范围就是我们常见的[0, 255]了，本文后续部分也会使用这个 RGB 范围。RGB 色彩空间的模型如下：

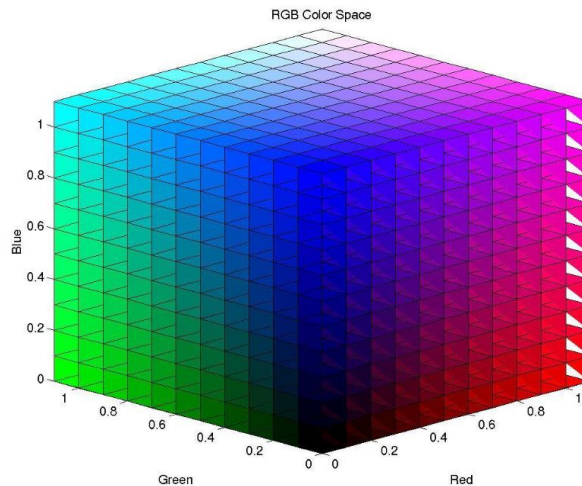


图 2.2-1 RGB 色彩空间模型

在 RGB 色彩空间下，颜色的色度，亮度，饱和度等信息无法区分，这在处理中是相当麻烦的，对于本项目，需要处理的是颜色的色度分量，而在 RGB 色彩空间下不好将不同色度区分开，因此引入 HSV 色彩空间模型。

2.2.2 HSV 色彩模型

HSV(Hue, Saturation, Value)是根据颜色的直观特性由 A. R. Smith 在 1978 年创建的一种颜色空间，也称六角锥体模型(Hexcone Model)。在 HSV 中，颜色是由色度 (Hue)，饱和度 (Saturation)，明度 (Value) 共同组成，色彩模型如下：

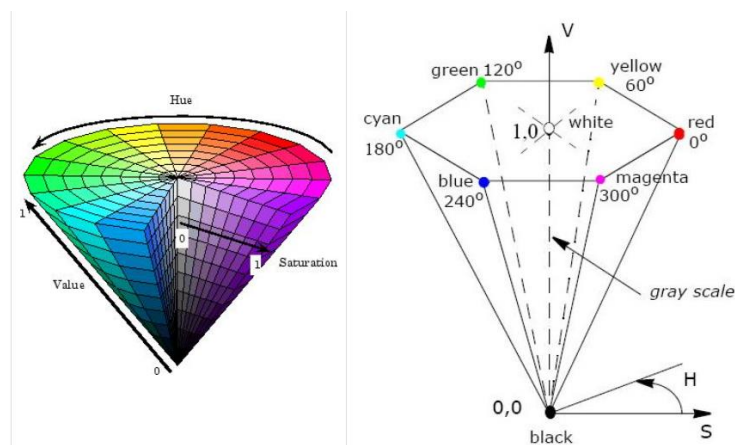


图 2.2-2 HSV 色彩空间模型

在 HSV 模型下，通过 H 分量即色度即可区分不同的颜色，从而通过设定不同的颜色阈值范围，达到区分颜色的目的，如下将介绍本模块中使用的 RGB 转 HSV 后通过改变 H 分量，再将改变 H 值后的 HSV 转化为 RGB，从而实现将一种颜色转化为另一种颜色。

RGB 转 HSV 公式

1. 明度（Value）计算

$$V = \max(R, G, B) \in [0, 255]$$

2. 饱和度计算（Saturation）计算

$$S = \begin{cases} 0 & \text{if } V = 0 \\ 255 \times \frac{V - \min(R, G, B)}{V} & \text{otherwise } \in [0, 255] \end{cases}$$

3. 色相（Hue）计算（分段函数）

$$H = \begin{cases} 60^\circ \times \frac{G - B}{\Delta} & \text{if } V = R \\ 60^\circ \times (2 + \frac{B - R}{\Delta}) & \text{if } V = G \\ 60^\circ \times (4 + \frac{R - G}{\Delta}) & \text{if } V = B \end{cases} \in [0^\circ, 360^\circ]$$

其中

$$\Delta = V - \min(R, G, B)$$

本项目中将红色和黄色转变为蓝色，再通过灰度模块提取蓝色色度分量，从而实现提取不同的颜色的目的，并且在本模块就区分出了四种颜色，在后续处理模块只需要区分出形状即可。另外为了后续方便同一参数位宽，我们将 h,s,v 的数值都映射在 0~255 的区间上，只需 6 个八位宽数据即可表示阈值。

2.2.3 二值化

在得到 HSV 数据后，只需对图像数据进行二值化，即可得到二值化数据，利于接下来的图像处理。

```
// 颜色定义参数
localparam COLOR_RED    = 2'b00;
localparam COLOR_BLUE   = 2'b01;
localparam COLOR_YELLOW = 2'b10;
localparam COLOR_BLACK  = 2'b11;

// 红色阈值 (注意红色在HSV色轮的两端)
localparam RED_H_MIN_LOW  = 8'd0;
localparam RED_H_MAX_LOW  = 8'd10;
localparam RED_H_MIN_HIGH = 8'd160;
localparam RED_H_MAX_HIGH = 8'd180;
localparam RED_S_MIN      = 8'd100;
localparam RED_S_MAX      = 8'd255;
localparam RED_V_MIN      = 8'd50;
localparam RED_V_MAX      = 8'd255;

// 蓝色阈值
localparam BLUE_H_MIN     = 8'd150;
localparam BLUE_H_MAX     = 8'd175;
localparam BLUE_S_MIN     = 8'd50;
localparam BLUE_S_MAX     = 8'd255;
localparam BLUE_V_MIN     = 8'd0;
localparam BLUE_V_MAX     = 8'd255;

// 黄色阈值
localparam YELLOW_H_MIN   = 8'd25;
localparam YELLOW_H_MAX   = 8'd35;
localparam YELLOW_S_MIN   = 8'd100;
localparam YELLOW_S_MAX   = 8'd255;
localparam YELLOW_V_MIN   = 8'd50;
localparam YELLOW_V_MAX   = 8'd255;

// 黑色阈值 (主要看V值)
localparam BLACK_H_MIN    = 8'd0;
localparam BLACK_H_MAX    = 8'd255;
localparam BLACK_S_MIN    = 8'd0;
localparam BLACK_S_MAX    = 8'd255;
localparam BLACK_V_MIN    = 8'd0;
localparam BLACK_V_MAX    = 8'd50;

// 中间信号
```

图 2.2-3 各颜色的 HSV 参数

通过比较阈值，在参数范围内的像素点赋值为白色，其他则是黑色。二值化后的图像如下：

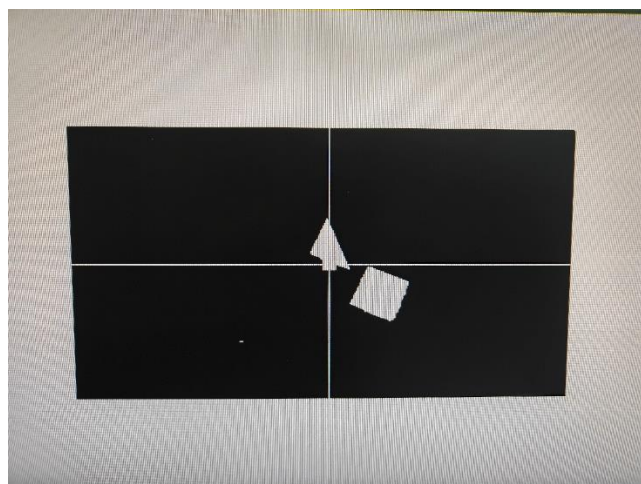


图 2.2-4 二值化图像

2.2.4 形态学滤波-开操作

图像开运算是图像依次经过腐蚀、膨胀处理后的过程。图像被腐蚀后，去除了噪声，但是也压缩了图像；接着对腐蚀过的图像进行膨胀处理，可以去除噪声，并保留原有图像。如下图所示：



图 2.2-5 开运算操作结果示例

1. 腐蚀

结构 A 被结构 B 腐蚀的定义为：

$$A \odot B = \{z | (B)z \subseteq A\}$$

可以理解为，移动结构 B，如果结构 B 与结构 A 的交集完全属于结构 A 的区域内，则保存该位置点，所有满足条件的点构成结构 A 被结构 B 腐蚀的结果。

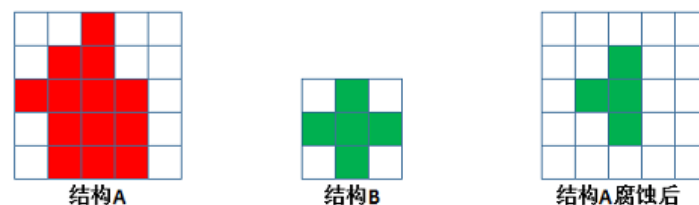


图 2.2-6 开运算腐蚀操作示例

2. 膨胀

结构 A 被结构 B 膨胀的定义为：

$$A \oplus B = \{z | (B^c)z \cap A \neq \emptyset\}$$

可以理解为，将结构 B 在结构 A 上进行卷积操作，如果移动结构 B 的过程中，与结构 A 存在重叠区域，则记录该位置，所有移动结构 B 与结构 A 存在交集的位置的集合为结构 A 在结构 B 作用下的膨胀结果。

图示中红色框内的区域表示结构 A 在结构 B 的作用下膨胀的结果：



图 2.2-7 开运算膨胀操作示例

2.2.5 目标识别

目标识别参考 b 站大磊老师识别纸钞的思路，目标检测逻辑采用动态边界框扩展算法，通过逐像素扫描和状态机控制实现实时多目标跟踪：系统在每帧开始时初始化所有目标边界框，当检测到有效像素(per_img_Bit=1)时，首先检查该像素坐标是否超出所有现有目标的扩展边界区域（边界外扩 MIN_DIST 像素），若超出则创建新目标（如检测到(100,80)像素时建立初始边界框）；对于已关联的目标，系统动态更新其边界框范围（如当发现更靠右的(150,120)像素时，将右边界从 100 扩展到 150），同时通过三帧流水线机制确保稳定的目标跟踪——首帧完成目标发现和边界确定，中间帧进行垂直投影面积计算，末帧执行形状分类和结果输出。通过(x_cnt,y_cnt)坐标计数器实现扫描同步，利用 target_pos 寄存器组维护各目标的实时边界信息（包含 40 位边界坐标和 1 位有效标志），并采用 vsync_cnt 状态机控制三阶段处理流程，最终输出包含形状编码、边界坐标和面积特征的 42 位结构化数据。

目标识别流程图：

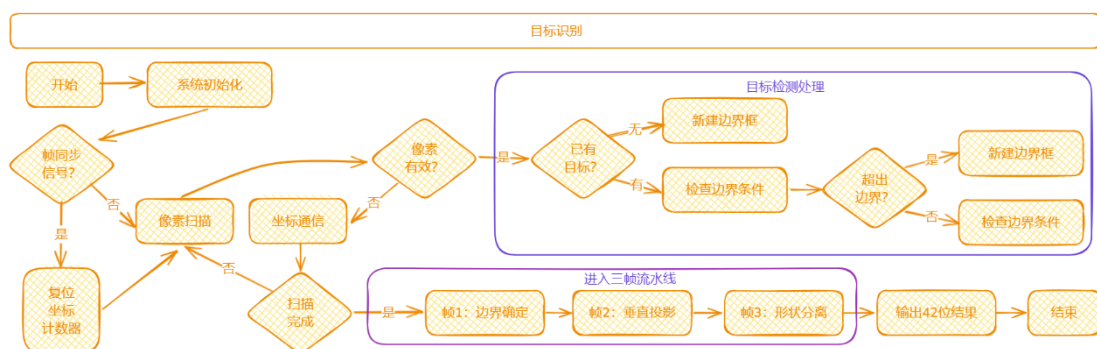


图 2.2-8 开目标识别流程图

得到四个 target_out 后传给数据处理模块，找到 req_shape 并输出目标的实际中心坐标给机械臂处理模块。

2.2.6 角度信息获取与坐标转换

对于三角形与正方形角度偏移的处理。我们使用记录物块边框坐标和扫描的第一个像素点的 x 坐标与最后一点的 x 坐标。如图：

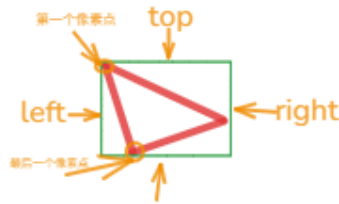


图 2.2-9 对于三角形偏移处理

对于正方形角度处理较为简单，如图：

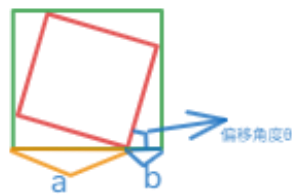


图 2.2-10 对于正方形偏移处理

计算公式如下

$$b = x_{\text{right}} - x_{\text{last}}$$

$$a = x_{\text{last}} - x_{\text{left}}$$

$$\theta = \arctan \frac{b}{a}$$

对于三角形的处理较为复杂，三角形的旋转过程有 16 中状态，其中 8 中状态为特定状态，8 钟状态为中间可变状态。如下图所示：

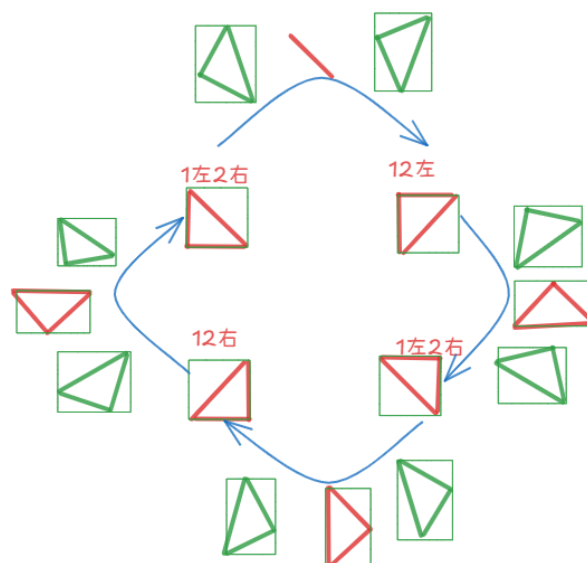


图 2.2-11 三角形 16 种偏移状态

这里只需要考虑其中八种中间可变状态，即图中绿色所示状态，然后进行归纳，处理只需将各个状态向距离最近的稳定状态旋转即可。如下图：

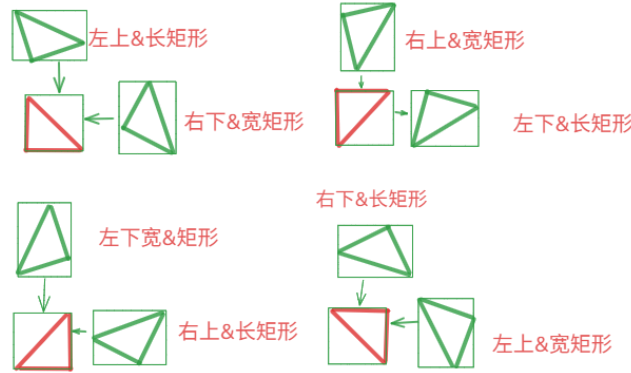


图 2.2-12 三角形偏移状态特定 8 种处理

处理只需将各个状态向距离最近的稳定状态旋转即可。代码如下：

```
2'b00: begin // 三角形处理 (完整8种情况)
// === 情况1: 左下直角 (宽矩形+起点在左+终点在右) ===
if (at_left_first_x && at_right_last && is_wide) begin
angle_mode <= 2'b00; // arctan((last_x-first_x)/height) 顺时针归位
shape_state <= 2'b00; // 左下直角
b <= last_x - first_x; // 底边长度
a <= height; // 高度
data_valid <= 1'b1;
end

// === 情况2: 左上直角 (高矩形+起点在左+终点在右) ===
else if (at_left_first_x && at_right_last && !is_wide) begin
angle_mode <= 2'b11; // 45°-arctan((last_x-first_x)/height) 逆时针归位
shape_state <= 2'b00; // 左下直角
b <= last_x - first_x;
a <= height;
data_valid <= 1'b1;
end

// === 情况3: 左上直角 (宽矩形+起点在左+终点在左) ===
else if (at_left_first_x && at_left_last && is_wide) begin
angle_mode <= 2'b01; // arctan((first_x-last_x)/height) 逆时针归位
shape_state <= 2'b01; // 左上直角
b <= first_x - last_x;
a <= height;
data_valid <= 1'b1;
end

// === 情况4: 左上直角 (高矩形+起点在右+终点在左) ===
else if (at_right_first_x && at_left_last && !is_wide) begin
angle_mode <= 2'b10; // 45°-arctan((last_x-first_x)/height) 顺时针归位
shape_state <= 2'b01; // 左上直角
b <= last_x - first_x;
a <= height;
data_valid <= 1'b1;
end
end
```

图 2.2-13 三角形偏移状态处理代码

对于坐标转换，再获取目标物块外围矩形的坐标的后计算出中心坐标，按照比例缩放再加上机械臂坐标系和图像坐标系的偏移距离即可得到实际坐标：

```
assign real_x = valid ? 0-(((temp_x-IMG_CENTER_X)*X_SCALE)>>16) : 16'd0;
assign real_y = valid ? (((temp_y-IMG_CENTER_Y)*Y_SCALE)>>16)+Y_OFFSET : 16'd0;
```

图 2.2-14 得到实际坐标

2.3 机械臂控制

2.3.1 逆运动学解算

得到实际坐标后，需要进行逆运动学解算，因为机械臂吸取时末端保持与水平面垂直效果较好，这为机械臂几何逆解提供了方便.

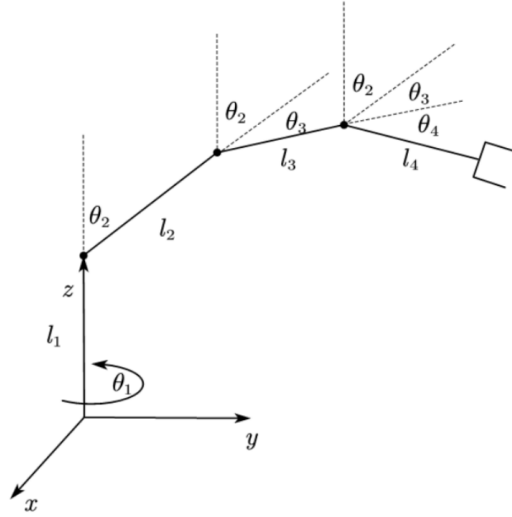


图 2.3-1 反解推导坐标系 1

由于末端与水平面垂直，有条件角度 2+角度 2+角度 4 的和为 90 度，于是运动模型可以变成下面这副图：

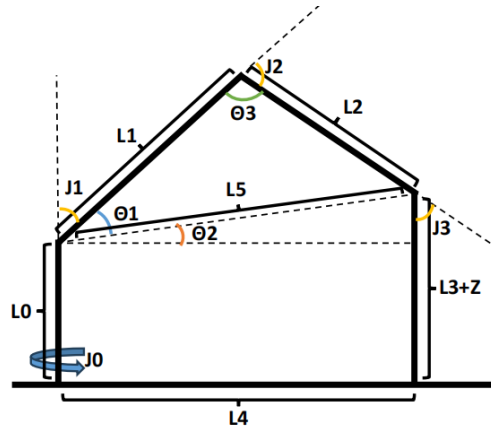


图 2.3-2 反解推导坐标系 2

利用三角性质，可以得到下面的表达式。计算过程如下：

$$L_4 = \sqrt{x^2 + y^2}, L_5 = \sqrt{(L_3 + z - L_0)^2 + L_4^2}$$

$$\cos \theta_1 = \frac{L_1^2 + L_5^2 - L_2^2}{2L_1L_5}, \cos \theta_3 = \frac{L_1^2 + L_2^2 - L_5^2}{2L_1L_5}$$

$$\theta_1 = \arccos(\cos \theta_1), \theta_2 = \arccos\left(\frac{L_4}{L_5}\right), \theta_3 = \arccos(\cos \theta_3)$$

$$j_0 = \arccos\left(\frac{y}{L_4}\right), j_1 = 90^\circ - \theta_1 - \theta_2, j_2 = 180^\circ - \theta_3, j_3 = \theta_1 + \theta_2 + \theta_3 - 90^\circ$$

整个模块只需要使用两个 IP 核：DIV 和 sqrt。其中 arccos 与 atan 的计算通过查找表的方式实现。

▼ Inverse_kinematics_ak:u_ik	1584 (514)	660 (430)	0 (0)	24576	3
> Acos_Rom:Acos_Rom	0 (0)	0 (0)	0 (0)	24576	3
> DIV:DIV	812 (0)	200 (0)	0 (0)	0	0
> lpm_mult:Mult0	0 (0)	0 (0)	0 (0)	0	0
> lpm_mult:Mult1	0 (0)	0 (0)	0 (0)	0	0
> lpm_mult:Mult2	0 (0)	0 (0)	0 (0)	0	0
> lpm_mult:Mult3	0 (0)	0 (0)	0 (0)	0	0
> lpm_mult:Mult4	0 (0)	0 (0)	0 (0)	0	0

图 2.3-3 资源占用情况

逆解需要 1584 个逻辑单元

2.3.2 仓库位置机械臂关节对应角度的存储

为了节省计算量，再机械臂吸取到物块后，我们采用将目标仓库位置机械臂关节对应角度采用查表表的形式存储，24 个槽位，每个槽位对应的角度有四个，每个角度（扩大十倍）使用 11 位宽有符号数储存。

只需输入目标仓库的槽位的索引即得到相应的旋转角度。

```

-- 机械臂控制ROM数据（严格12位有符号数版本）
-- 格式：{J3[11:0], J2[11:0], J1[11:0], J0[11:0]}
-- 每行12个十六进制字符（无分隔符），表示48位数据
WIDTH = 48;
DEPTH = 24;
ADDRESS_RADIX = DEC;
DATA_RADIX = HEX;

CONTENT BEGIN
-- Y=0
0 : ABC05742228F; --j0: -1348 (0xABC) j1: 86 (0x057) j2: 1057 (0x422) j3: 655 (0x28F)
1 : B210CB38C2B1; --j0: -1247 (0xB21) j1: 202 (0x0CB) j2: 907 (0x38C) j3: 689 (0x2B1)
2 : B651522BF2F7; --j0: -1178 (0xB65) j1: 337 (0x152) j2: 703 (0x2BF) j3: 759 (0x2F7)
3 : B9620618537D; --j0: -1130 (0xB96) j1: 518 (0x206) j2: 388 (0x185) j3: 892 (0x37D)

-- Y=1
4 : B4BFEE492288; --j0: -1205 (0xB4B) j1: -17 (0xFEE) j2: 1169 (0x492) j3: 647 (0x288)
5 : B9F07C3F5297; --j0: -1120 (0xB9F) j1: 124 (0x07C) j2: 1012 (0x3F5) j3: 663 (0x297)
6 : BD210D32B2D0; --j0: -1069 (0xBD2) j1: 269 (0x10D) j2: 810 (0x32B) j3: 719 (0x2D0)
7 : BF41B621633C; --j0: -1035 (0xBF4) j1: 438 (0x1B6) j2: 533 (0x216) j3: 828 (0x33C)

-- Y=2
8 : C1AFAB4CE28F; --j0: -998 (0xC1A) j1: -84 (0xFAB) j2: 1229 (0x4CE) j3: 654 (0x28F)
9 : C3D05042A28E; --j0: -963 (0xC3D) j1: 79 (0x050) j2: 1065 (0x42A) j3: 654 (0x28E)
10 : C500EA35F2BF; --j0: -944 (0xC50) j1: 234 (0x0EA) j2: 862 (0x35F) j3: 702 (0x2BF)
11 : C5C192254322; --j0: -932 (0xC5C) j1: 402 (0x192) j2: 595 (0x254) j3: 801 (0x322)

-- Y=3
12 : D06FAB4CE28F; --j0: -761 (0xD06) j1: -84 (0xFAB) j2: 1229 (0x4CE) j3: 654 (0x28F)
13 : CE305042A28E; --j0: -796 (0xCE3) j1: 79 (0x050) j2: 1065 (0x42A) j3: 654 (0x28E)
14 : CD00EA35F2BF; --j0: -815 (0xCD0) j1: 234 (0x0EA) j2: 862 (0x35F) j3: 702 (0x2BF)
15 : CC4192254322; --j0: -827 (0xCC4) j1: 402 (0x192) j2: 595 (0x254) j3: 801 (0x322)

-- Y=4
16 : DD5FEE492288; --j0: -554 (0xDD5) j1: -17 (0xFEE) j2: 1169 (0x492) j3: 647 (0x288)
17 : D8107C3F5297; --j0: -639 (0xD81) j1: 124 (0x07C) j2: 1012 (0x3F5) j3: 663 (0x297)
18 : D4E10D32B2D0; --j0: -690 (0xD4E) j1: 269 (0x10D) j2: 810 (0x32B) j3: 719 (0x2D0)
19 : D2C1B621633C; --j0: -724 (0xD2C) j1: 438 (0x1B6) j2: 533 (0x216) j3: 828 (0x33C)

-- Y=5
20 : E6405742228F; --j0: -411 (0xE64) j1: 86 (0x057) j2: 1057 (0x422) j3: 655 (0x28F)
21 : DFF0CB38C2B1; --j0: -512 (0xDFF) j1: 202 (0x0CB) j2: 907 (0x38C) j3: 689 (0x2B1)
22 : DBB1522BF2F7; --j0: -581 (0xDBB) j1: 337 (0x152) j2: 703 (0x2BF) j3: 759 (0x2F7)
23 : D8A20618537D; --j0: -629 (0xD8A) j1: 518 (0x206) j2: 388 (0x185) j3: 892 (0x37D)
END;

```

图 2.3-4 机械臂关节对应角度 mif 文件

2.3.3 减少误差设计

对于机械臂误差设计，总的来说影响因素有：

摄像头图像获取坐标误差：使用的 OV5640 视觉传感器的镜头存在一定畸变，但在实际测试坐标时发现图像处理得到的坐标与实际坐标误差范围能够控制在 0.5mm 以内。对于本设计这是一个可以接受的误差，对于准确的影响较小。

逆解误差：使用 Verilog 语言实现的逆解存在开平方误差，反三角函数误差，除法误差，但实际通过编写仿真文件测试，发现误差极小，将使用 Verilog 语言解算出来的结果与 c 语言解算的结果相比较，发现最大误差范围为 1° 左右，对于选择的理论精度为 0.7° 的舵机来说可以忽略不计。

部分坐标逆解对比：

坐标 (x,y,z) 单位 (mm)	verilog 逆解 (0.1°)	c 语言逆解 (1°)	平均误差 (1°)
(112, 240, 40)	(649, 512, 401, 887)	(65, 52, 39, 90)	0.575
(112, 195, 40)	(600, 336, 716, 748)	(60, 34, 70, 76)	0.8
(112, 150, 40)	(532, 203, 910, 687)	(53, 20, 91, 69)	0.2
(112, 105, 40)	(429, 86, 1063, 651)	(43, 9, 106, 66)	0.425
(76, 105, 40)	(573, -17, 1175, 642)	(57, -2, 117, 65)	0.475

舵机本身与机械臂结构产生的误差：一是受舵机本身设计中存在的误差，约为 0.7°，且与舵机品质高度相关。二是因为机械臂物理结构所受重力产生的误差，较重的机械臂结构与机械臂的伸出结构所产生的惯性出现的不可避免的误差，三是机械臂运动过程中，力学结构发生变化导致的误差。

由于本次地图较大如果将机械臂放置在整个地图的中心，理论上地图上需要吸取物块离机械臂中心的距离在 265mm 左右。

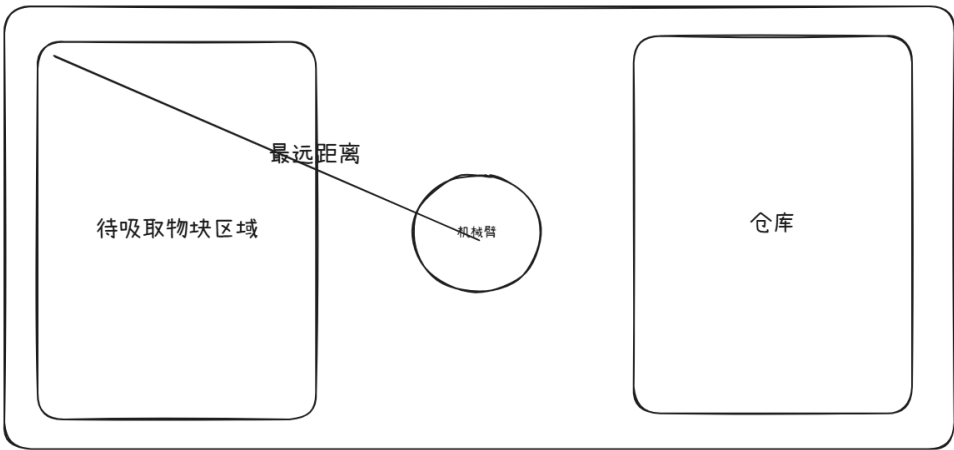


图 2.3-5 机械臂与最远位置

我们在淘宝使用所购置的机械臂支架组装后并不能达到最远距离，因此我采用在关节链接处使用同住连接达到延长机械臂长度的效果

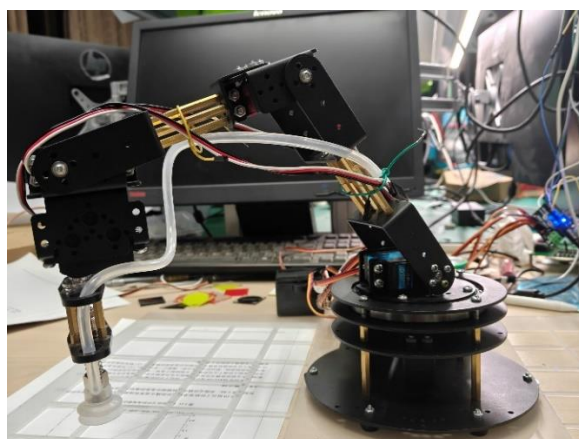


图 2.3-6 机械臂图

由于加长的大臂与小臂的长度，随之而来就是导致整个机械臂因为重力导致大臂关节与小臂关节会不受舵机角度控制的下降一段角度，会导致在机械臂运动到指定位置后距离目标物块还存在一段距离不够。

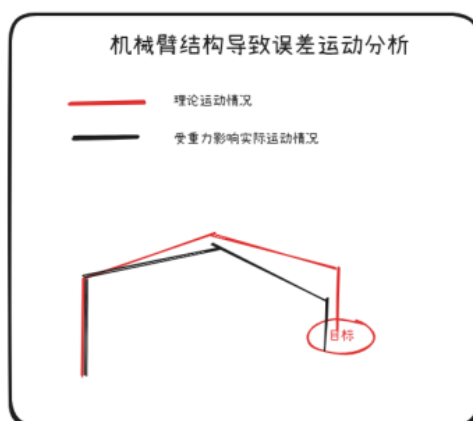


图 2.3-7 运动误差分析

但也是由于机械臂的大臂和小臂加长，使得在运动到不同距离的时候，大臂和小臂的误差角度的动态性减少，几乎都是最大误差角度，我们实测发现大臂误差角度为 5 度左右，小臂关节误差角度在 3 度左右。补偿大臂和小臂误差后会使得末端关节的水平距离与目标物块一致，但是末端吸盘的角度与目标物块的中心位置可能有误差，如下图所示

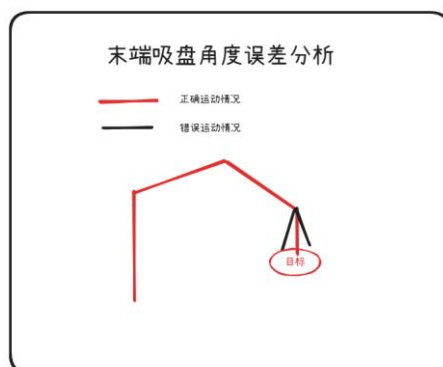


图 2.3-8 末端吸盘误差分析

整个误差需要根据距离动态补偿代码实现如下

```
// 动态调整补偿值 (根据距离)
always@(posedge Clk or negedge Rst_n) begin
    if(!Rst_n)
        adjust2 <= 'd0;
    else if(cnt_s == 'd4) begin
        if(L4 >= 230)
            adjust2 <= 'd0;
        else if(L4>=185)
            adjust2 <= 'd30;
        else if(L4>=150)
            adjust2 <= 'd30;
        else adjust2 <= 'd60;
    end
end
```

图 2.3-9 距离动态补偿代码

逆解最终角度输出

```
end
else if(valid_a_r[2]) begin
    J0 <= J0r+20;
    J1 <= J1r-30;
    J2 <= J2r-50;//50
    J3 <= J3r+adjust2; // 应用动态调整值
end
end
```

图 2.3-10 逆解最终角度输出代码

2.3.4 角度偏移处理

对于末端吸盘转动角度设计，在得到偏置角度后，将吸取物块旋转至目标仓库放置的角度后，还需考虑由于机械臂地盘旋转导致的角度偏差，图解如下

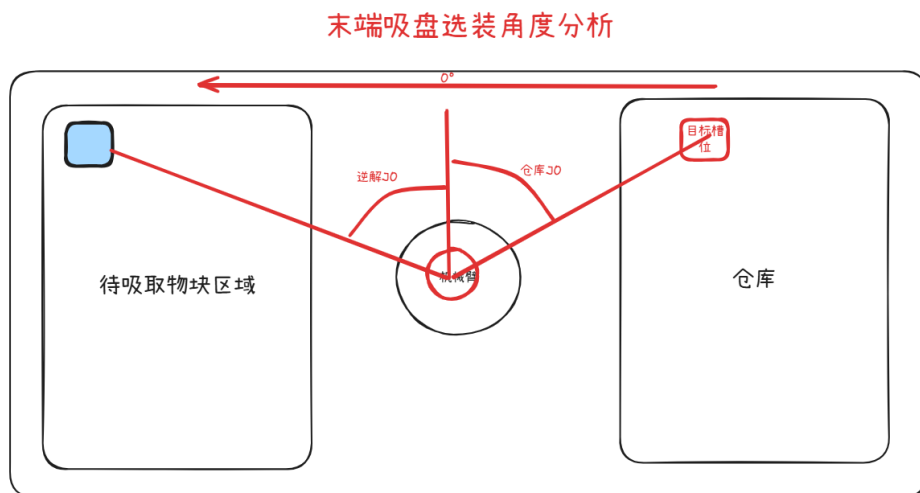


图 2.3-11 末端吸盘选装角度分析

可知末端选装角度为逆解 J0 减去仓库 J0 的相反数代码实现如下

```

always @(*) begin
    // 1. 计算原始角度差 (自动处理溢出)
    j4_angle_raw = target_j0 - store_j0;

    // 2. 将角度差映射到[0, 360)范围
    if ($signed(j4_angle_raw) < 0)
        j4_angle_raw = j4_angle_raw + ANGLE_360;
    else if ($signed(j4_angle_raw) >= ANGLE_360)
        j4_angle_raw = j4_angle_raw - ANGLE_360;

    // 3. 限制到[0, 180]范围
    if ($signed(j4_angle_raw) > ANGLE_180)
        j4_angle_raw = ANGLE_360 - j4_angle_raw; // 取镜像角度

    // 4. 取负数得到[-180, 0]范围
    j4_angle_final = -j4_angle_raw;
end

```

图 2.3-12 末端吸盘选装代码

2.3.5 单个吸取状态机设计

对于单个目标吸取，我们设计的是先让机械臂运行到目标物块的上方 4 厘米处停留 1s 等待机械臂稳定后再运行到 2 厘米的位置并开启停留 1s 在运动到高度 0 厘米位置进行吸取 3s。

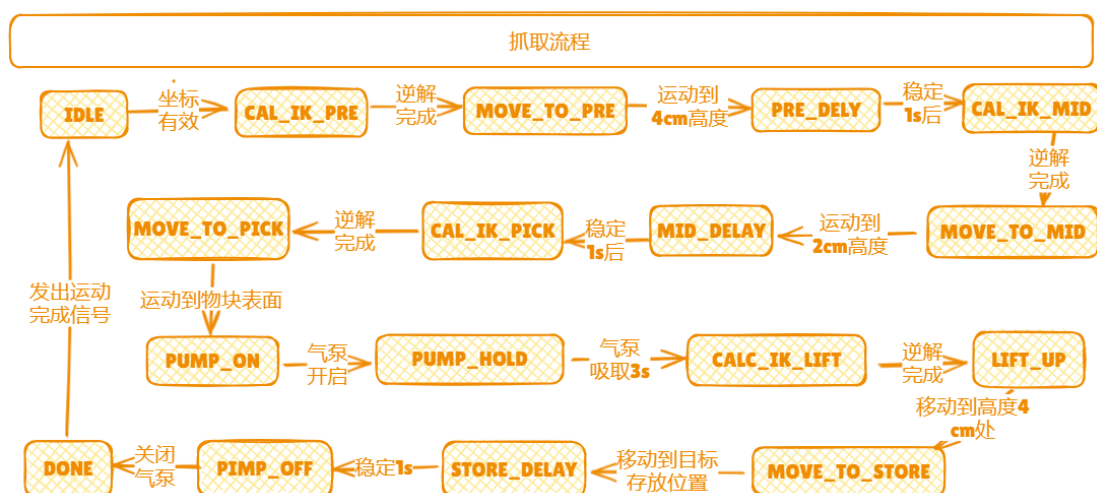


图 2.3-13 抓取流程图

在实际测试过程，我们发现机械臂在完成一次运动从仓库位置运动到下一个目标位置的过程中，由于各个机械臂并行运动，如果末端反应相较次末端或者大臂关节速度较慢，可能会出现机械臂碰到地面的情况。

为了避免这种钟情况我们测试得出结论目标准备抓取准备位置的高度和运动到仓库存储位置上方高度为 4cm 的时候，不会出现机械臂碰地的情况，另外机械臂各个关节的速度也有影响，末端关节的运动速度应该是最快。向下关节运动速度依次减少。这样设计避免复杂的运动轨迹规划，实现使用较为简单的轨迹控制实现本项目的正确吸取任务。

2.4 任务控制

2.4.1 目标位置信息传递

为了节省目标仓库的 4 息储存使用 rom 查找表实现，每个槽位的信息包括：位置索引，颜色，形状，放置角度。可以使用一个 7 位宽，深度为 24 的 rom 实现：

```
DEPTH = 24;          -- 24 个存储单元
WIDTH = 7;           -- 每个单元 7 位宽
ADDRESS_RADIX = DEC; -- 地址采用十进制表示
DATA_RADIX = BIN;    -- 数据采用二进制表示

CONTENT
BEGIN
    -- 地址：数据 (bit6-5:颜色, bit4-3:形状, bit2-1:角度, bit0:需放置标志)
    -- 颜色编码：00=红 01=蓝 10=黄 11=黑
    -- 形状编码：00=三角 01=六边 10=圆形 11=方形

    0 : 0000000; -- [空位置]
    1 : 0000000; -- [空位置]
    2 : 0000000; -- [空位置]
    3 : 0000000; -- [空位置]

    4 : 0000000; -- [空位置]
    5 : 0000000; -- [空位置]
    6 : 0000000; -- [空位置]
    7 : 0000000; -- [空位置]

    8 : 0011001; -- [空位置]
    9 : 0010001; -- [空位置]
    10 : 0001001; -- [空位置]
    11 : 0000001; -- [空位置]

    12 : 0111001; -- [空位置]
    13 : 0110001; -- [空位置]
    14 : 0101001; -- [空位置]
    15 : 0100001; -- [空位置]

    16 : 1111001; -- 蓝色(01) 三角(00) 角度 00 需放置(1)
    17 : 1110001; -- 蓝色(01) 六边(01) 角度 01 需放置(1)
    18 : 1101001; -- [空位置]
    19 : 1100001; -- [空位置]

    20 : 1011001; -- 红色(00) 方形(11) 角度 00 需放置(1)
```

21 : 1010001; -- 红色(00) 圆形(10) 角度 00 需放置(1)
22 : 1001001; -- 红色(00) 六边(01)角度 00 需放置(1)
23 : 1000001; -- 红色(00) 三角(00) 角度 00 需放置(1)

END;
通过查询各个位置的信息依次发出工作请求。状态转移图如下：

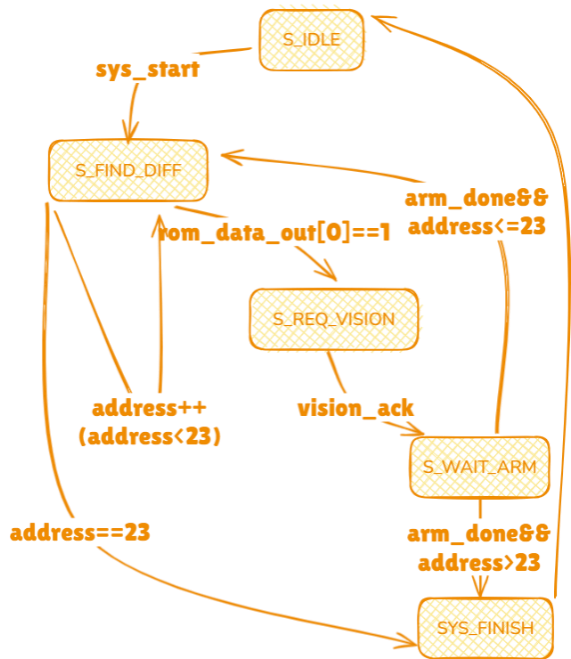


图 2.4-1 工作状态转移图

仿真波形如下：

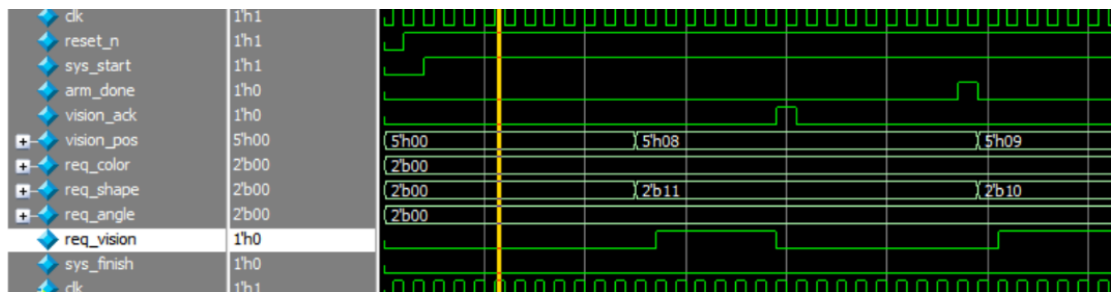


图 2.4-2 仿真波形

可以看到能够将提前存储好的位置信息正确发出，从而正确的完成任务。

2.4.2 跨时钟域处理

图像处理和机械臂运动沟通信号的跨时钟域处理，由于图像处理模块使用的时钟是外部 ov5640 的 pclk 端口的时钟，机械臂运动的时钟是系统时钟，二者进行通信时候需要进行跨时钟处理，做法是将图像处理产生的坐标有效信号延长 7 个图像处理时钟，机械单次运动完场的信号同样延长 7 个系统时钟周期，从而保证整个系统工作时得稳定性。

图像处理模块发送 7 周期的应答信号：

```

ACK_MATCH: begin
    stab timer <= 0;
    // 遍历目标寻找匹配项
    for (i = 0; i < 4; i=i+1) begin
        if(target_valid[i] && (shape_in[i] == target_shape)) begin
            temp_x <= center_x[i];
            temp_y <= center_y[i];
            vision_ack <= 1'b1;
            valid <= 1'b1; // 立即标记有效
            ack_state <= ACK_SEND;
        end
    end
end
// 应答发送状态
ACK_SEND: begin
    if(hold_cnt<7)
    begin
        hold_cnt<=hold_cnt+1'b1;
    end
    else ack_state <= ACK_IDLE;
end

```

图 2.4-3 发送 7 周期的应答信号

机械臂处理模块发出 7 周期的单次运动结束信号:

```

end
DONE_HOLD: begin
    if(hold_cnt<3'd4)
        hold_cnt<=hold_cnt+1'b1;
    else state <= IDLE;
end
//state <= IDLE;
default: state <= IDLE;

```

图 2.4-4 发送 7 周期的单次运动结束信号

2.5 上位机设计与控制

2.5.1 软件架构设计

本软件基于 Qt 框架开发，主要用于对机械臂进行控制测试和 hsv 调参，支持三种工作模式切换，具备滑块参数调节、十六进制 / 十进制数据显示、串口通信配置及数据收发等功能。上位机界面如图：



图 2.5-1 上位机 UI 界面

2.5.2 核心功能实现

串口通信：支持自动扫描端口、标准波特率选择，固定配置为 8N1（8 数据位、无校验、1 停止位），打开串口后启用模式切换。串口打开流程：

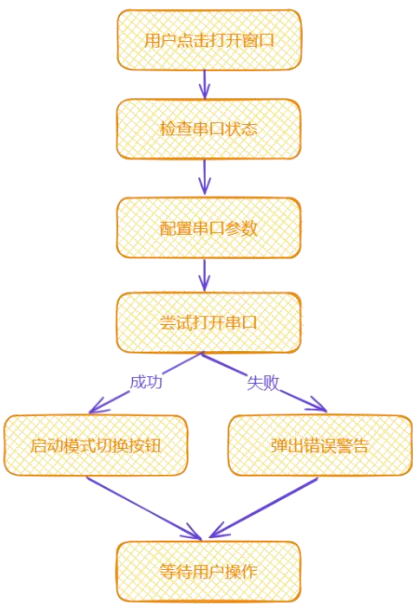


图 2.5-2 串口打开流程图

参数调节：前 5 滑块（127~127）控制舵机（负数转补码，显示十六进制），后 6 滑块（0~255）控制 HSV（直接转无符号字节，显示十进制）。

模式切换：三种模式循环切换（发送区/舵机/HSV），对应激活不同组件并禁用其他组件。模式切换流程：

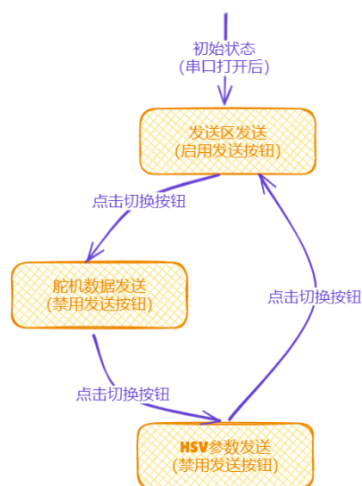


图 2.5-3 模块切换流程图

2.5.3 关键技术实现

十六进制处理：支持用户通过发送区输入十六进制字符串（如 00 FF A0），需转换为字节数组发送；同时，滑块值生成的指令也需按十六进制格式封装。输入字符串去空格、校验长度（偶数）、逐字节转换，错误时提示格式异常。

代码实现：

```
QByteArray MainWindow::hexStringToByteArray(const QString &hexStr) {
    QString cleanedStr =
hexStr.trimmed().remove(QRegularExpression("\\s+"));
    if (cleanedStr.length() % 2 != 0) { /* 长度校验 */ }
    for (int i = 0; i < cleanedStr.length(); i += 2) {
        QString byteStr = cleanedStr.mid(i, 2);
        char byte = static_cast<char>(byteStr.toInt(&ok, 16)); /* 逐字节转换
*/
        data.append(byte);
    }
    return data;
}
```

模式状态机管理：通过枚举类型和状态更新函数实现三种模式的循环切换，控制 UI 控件的可用状态，避免用户误操作。

代码实现：

```
void MainWindow::on_switchModeBtn_released() {
    currentMode = static_cast<Mode>(((static_cast<int>(currentMode) + 1) %
3);
    updateModeUI();
}
```

```

    }

    void MainWindow::updateModeUI() {
        switch (currentMode) {
            case Mode::Mode1: ui->openPortBtn_2->setText("发送区发送");
            ui->sendBtn->setEnabled(true); break;
            case Mode::Mode2: ui->openPortBtn_2->setText("舵机数据发送");
            ui->sendBtn->setEnabled(false); break;
            case Mode::Mode3: ui->openPortBtn_2->setText("HSV 参数发送");
            ui->sendBtn->setEnabled(false); break;
        }
    }
}

```

2.5.4 错误处理与异常机制

串口错误：打开失败时提示端口名称及错误信息（如权限不足、端口占用），波特率不匹配需手动重新配置。 **数据格式错误：**十六进制长度奇数或包含无效字符时弹出警告，终止发送操作。

2.5.5 使用指南

快速操作：

连接设备→选择端口/波特率→打开串口→切换模式→调节参数或输入数据。

模式要点：

发送区模式：支持 ASCII/十六进制，勾选复选框切换格式。

舵机模式：滑块值对应舵机角度（127~127），标签显示十六进制值。

HSV 模式：滑块值直接映射为 0~255 的无符号数，标签显示十进制值。

操作流程如图：

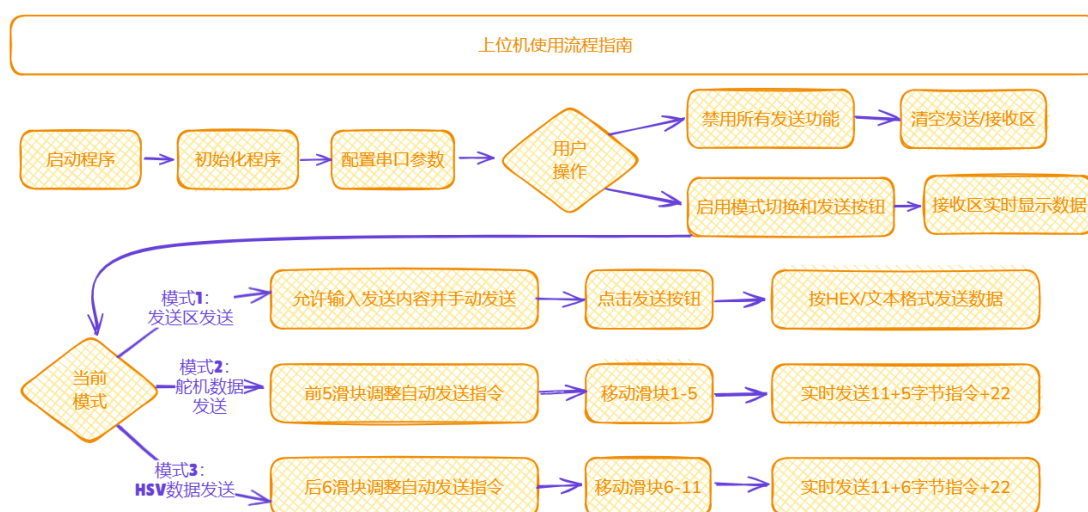


图 2.5-4 上位机使用流程图

3 性能分析

3.1 目标锁定时间

3.1.1. 图像采集与预处理耗时分析

系统首先接收 RGB888 格式的原始图像（640x480@60fps），通过 RGB2HSV 模块在 11 个时钟周期（148.2ns）内完成色彩空间转换，提取色调（H）、饱和度（S）、明度（V）分量。随后 HSV 二值化模块在 2 周期（26.94ns）内根据目标颜色阈值生成二值图像（如黄色物体对应 $H=50-80$, $S>135$ ）。

3.1.2. 形态学处理耗时分析

二值图像先后经过腐蚀（6 周期/80.82ns）和膨胀（6 周期/80.82ns）处理：腐蚀使用 3x3 核消除细小噪点，通过移位寄存器和两级与操作实现；膨胀同样采用 3x3 核恢复目标尺寸，通过两级或操作完成；此阶段共消耗 12 周期（161.64ns），有效分离粘连目标并平滑边缘。

3.1.3. 目标检测与形状识别耗时分析

shape_recognition 模块启动三帧检测流程：

首帧（N 帧）：检测目标初始位置，记录边界框（left/right/top/bottom）；

次帧（N+1 帧）：跟踪目标运动轨迹，更新边界坐标；

末帧（N+2 帧）：计算垂直投影面积（通过 4 个专用 BRAM 存储累加值），根据面积阈值分类形状（正方形>4000 像素，圆形>3100 像素等）；

该阶段占系统 95%以上的延迟，确保在复杂背景下稳定识别。

3.1.4. 坐标转换与输出耗时分析

中心计算：取边界框均值获得像素坐标（如(320,240)）；

X 坐标： $(x-320) \times 0.46$ （Q16 定点数乘法后截断为整数 mm）；

Y 坐标： $(y-240) \times 0.42 + 172$ （含机械坐标系偏移补偿；

应答握手：通过 valid 和 vision_ack 信号同步外部设备。

3.1.5. 整体分析耗时分析

Img_pro 模块的时序如下：

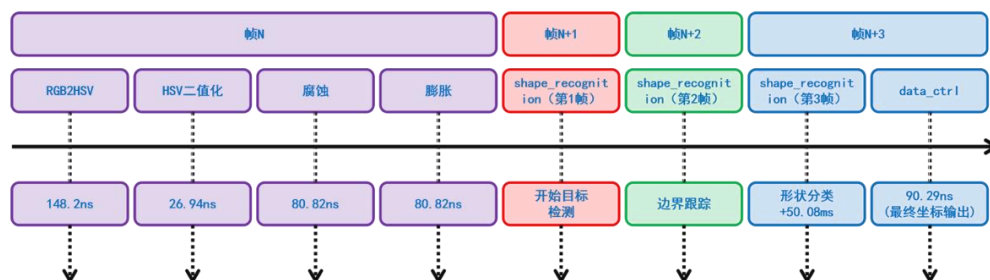


图 3.1-1 Img_pro 模块时序

各模块时间消耗如下

处理阶段	延迟 (T: 周期)	关键说明
RGB转HSV (RGB2HSV)	11T (148.2ns)	含除法器流水线
HSV二值化 (HSV_binsrixstion)	2T (26.94ns)	并行阈值比较
腐蚀处理 (Bit_Erosion_Detector)	6T (80.82ns)	3x3核运算+行缓存
膨胀处理 (Bit_Dilation_Detector)	6T (80.82ns)	与腐蚀对称设计
形状识别 (shape_recognition)	3帧+4T	三帧检测机制+实时计算 (约50.08ms)
坐标转换 (data_ctrl)	7T (94.29ns)	Q16定点数运算
总和 (流水线化)	≈50.1ms	受限于shape_recognition的三帧检测

图 3.1-2 各模块时序

总结：从物块进入视野到输出稳定坐标需要约 50.1 毫秒，

3.2 吸取准确度与仓库放置准确度

再设置好各中误差补偿参数后后，我们的机械臂吸取准确度可以达到百分之 90 左右，仓库放置准确度再百分之 70 左右

3.3 移动速度

通过使用计数器计数控制移动速度，当计数到一定数值时候，控制舵机的 pwm 波的占空比步进 1，由此达到控制速度的效果。代码实现如下：

```
// 运动控制
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        speed_cnt <= 0;
        current_angle <= INIT_ANGLE;
    end
    else if (en && !angle_error) begin // 添加en使能控制
        if (current_angle == target_angle) begin
            speed_cnt <= 0;
        end
        else begin
            if (speed_cnt == INTERNAL_SPEED) begin
                speed_cnt <= 0;
                current_angle <= (current_angle < target_angle) ?
                    current_angle + 1 : current_angle - 1;
            end
            else begin
                speed_cnt <= speed_cnt + 1;
            end
        end
    end
end
```

图 3.2-1 计数器控制移速

经测试在各个舵机速度设置为如下，则能够使机械臂工作速度较快，且

运动过程中不出现碰撞：

```
parameter INTERNAL_SPEED = 200_000
parameter SPEED0 =INTERNAL_SPEED/3;
parameter SPEED1 =INTERNAL_SPEED/3;
parameter SPEED2 =INTERNAL_SPEED/3;
parameter SPEED3 =INTERNAL_SPEED/3;
parameter SPEED4 =INTERNAL_SPEED/4;
```

图 3.2-2 舵机速度设置

加上视觉处理时间和运动过程中状态的稳定时间（仓库稳定时间 2s，目标上方稳定时间 2s），处理十六个物块需要的时间在 4 分 30 秒左右。